

BACKEND WORKFLOW PATTERNS

Module 2 — Governed AI Feature Delivery

THE PROBLEM WE'RE SOLVING

LLM calls scattered across controllers quickly become
unmaintainable.

-
-
-
-

THE PROBLEM WE'RE SOLVING

LLM calls scattered across controllers quickly become unmaintainable.

- Unclear ownership of prompt logic
-
-
-

THE PROBLEM WE'RE SOLVING

LLM calls scattered across controllers quickly become unmaintainable.

- Unclear ownership of prompt logic
- Inconsistent validation
-
-

THE PROBLEM WE'RE SOLVING

LLM calls scattered across controllers quickly become unmaintainable.

- Unclear ownership of prompt logic
- Inconsistent validation
- Difficult testing and reuse
-

THE PROBLEM WE'RE SOLVING

LLM calls scattered across controllers quickly become unmaintainable.

- Unclear ownership of prompt logic
- Inconsistent validation
- Difficult testing and reuse
- Weak audit trails

THE PROBLEM WE'RE SOLVING

LLM calls scattered across controllers quickly become unmaintainable.

- Unclear ownership of prompt logic
- Inconsistent validation
- Difficult testing and reuse
- Weak audit trails

Now: design a layered, reusable backend workflow.

WHAT "GOOD" LOOKS LIKE IN THIS MODULE

-
-
-
-
-

WHAT "GOOD" LOOKS LIKE IN THIS MODULE

- Deterministic orchestration around model calls
-
-
-
-

WHAT "GOOD" LOOKS LIKE IN THIS MODULE

- Deterministic orchestration around model calls
- Prompt assets in code with version labels
-
-
-

WHAT "GOOD" LOOKS LIKE IN THIS MODULE

- Deterministic orchestration around model calls
- Prompt assets in code with version labels
- Pre-call and post-call validation gates
-
-

WHAT "GOOD" LOOKS LIKE IN THIS MODULE

- Deterministic orchestration around model calls
- Prompt assets in code with version labels
- Pre-call and post-call validation gates
- Gateway-mediated calls with trace metadata
-

WHAT "GOOD" LOOKS LIKE IN THIS MODULE

- Deterministic orchestration around model calls
- Prompt assets in code with version labels
- Pre-call and post-call validation gates
- Gateway-mediated calls with trace metadata
- Safe fallback path when validation fails

ANTI-PATTERN TO AVOID

```
// Controller calls model directly – no contract boundary
@Post("/extract")
async extract(@Body() body: any) {
  const result = await this.llm.invoke(body.text);
  return result;
}
```

-
-
-
-

ANTI-PATTERN TO AVOID

```
// Controller calls model directly – no contract boundary
@Post("/extract")
async extract(@Body() body: any) {
  const result = await this.llm.invoke(body.text);
  return result;
}
```

- No input validation contract
-
-
-

ANTI-PATTERN TO AVOID

```
// Controller calls model directly – no contract boundary
@Post("/extract")
async extract(@Body() body: any) {
  const result = await this.llm.invoke(body.text);
  return result;
}
```

- No input validation contract
- No prompt version tracking
-
-

ANTI-PATTERN TO AVOID

```
// Controller calls model directly – no contract boundary
@Post("/extract")
async extract(@Body() body: any) {
  const result = await this.llm.invoke(body.text);
  return result;
}
```

- No input validation contract
- No prompt version tracking
- No output schema enforcement
-

ANTI-PATTERN TO AVOID

```
// Controller calls model directly – no contract boundary
@Post("/extract")
async extract(@Body() body: any) {
  const result = await this.llm.invoke(body.text);
  return result;
}
```

- No input validation contract
- No prompt version tracking
- No output schema enforcement
- No trace or audit metadata

WORKFLOW LAYERING IN PRACTICE

-
-
-
-
-

WORKFLOW LAYERING IN PRACTICE

- Controller: request/response contract only
-
-
-
-

WORKFLOW LAYERING IN PRACTICE

- **Controller:** request/response contract only
- **Workflow service:** orchestration and decisions
-
-
-

WORKFLOW LAYERING IN PRACTICE

- **Controller:** request/response contract only
- **Workflow service:** orchestration and decisions
- **Prompt module:** versioned prompt assets
-
-

WORKFLOW LAYERING IN PRACTICE

- **Controller:** request/response contract only
- **Workflow service:** orchestration and decisions
- **Prompt module:** versioned prompt assets
- **Validators:** pre/post-call checks
-

WORKFLOW LAYERING IN PRACTICE

- **Controller:** request/response contract only
- **Workflow service:** orchestration and decisions
- **Prompt module:** versioned prompt assets
- **Validators:** pre/post-call checks
- **Gateway adapter:** model access and tracing

LAYER RESPONSIBILITIES: CONTROLLER AND WORKFLOW SERVICE

Layer	Owns	Avoids
Controller	Transport contract, auth context	Orchestration and prompt logic
Workflow service	Sequence, branching, retries, fallback	HTTP transport concerns

LAYER RESPONSIBILITIES: PROMPT MODULE AND VALIDATORS

Layer	Owns	Avoids
Prompt module	Task instructions, placeholders, version id	Business policy and branching
Validators	Schema and policy checks	Model invocation

LAYER RESPONSIBILITIES: GATEWAY ADAPTER

Layer	Owns	Avoids
Gateway adapter	Model routing, trace metadata, audit envelope	Domain decision logic

LAYER RESPONSIBILITIES: GATEWAY ADAPTER

Layer	Owns	Avoids
Gateway adapter	Model routing, trace metadata, audit envelope	Domain decision logic

Each layer has one job. Mixing them is where maintainability breaks down.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

1. Validate incoming request schema in controller.
- 2.
- 3.
- 4.
- 5.
- 6.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

1. Validate incoming request schema in controller.
2. Workflow builds prompt variables and context.
- 3.
- 4.
- 5.
- 6.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

1. Validate incoming request schema in controller.
2. Workflow builds prompt variables and context.
3. Gateway call includes trace id and prompt version.
- 4.
- 5.
- 6.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

1. Validate incoming request schema in controller.
2. Workflow builds prompt variables and context.
3. Gateway call includes trace id and prompt version.
4. Model output is parsed into expected schema.
- 5.
- 6.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

1. Validate incoming request schema in controller.
2. Workflow builds prompt variables and context.
3. Gateway call includes trace id and prompt version.
4. Model output is parsed into expected schema.
5. Post-call validator enforces schema and policy.
- 6.

REQUEST FLOW (MODULE 2 IMPLEMENTATION PATH)

1. Validate incoming request schema in controller.
2. Workflow builds prompt variables and context.
3. Gateway call includes trace id and prompt version.
4. Model output is parsed into expected schema.
5. Post-call validator enforces schema and policy.
6. Return accepted output or safe fallback response.

PROMPT ASSETS IN CODE

-
-
-
-

PROMPT ASSETS IN CODE

- Store prompt templates as code artefacts, not hidden strings.
-
-
-

PROMPT ASSETS IN CODE

- Store prompt templates as code artefacts, not hidden strings.
- Attach version metadata for comparison and rollback.
-
-

PROMPT ASSETS IN CODE

- Store prompt templates as code artefacts, not hidden strings.
- Attach version metadata for comparison and rollback.
- Parameterise fields explicitly — no implicit mutation.
-

PROMPT ASSETS IN CODE

- Store prompt templates as code artefacts, not hidden strings.
- Attach version metadata for comparison and rollback.
- Parameterise fields explicitly — no implicit mutation.
- Review prompt changes like behaviour changes: diff, approve, merge.

STRUCTURED OUTPUT CONTRACT

Field	Type	Rule
documentType	enum	Must be one of approved categories
confidence	number	0–1, required
entities	array	Required, max length policy-constrained
summary	string	Optional, length-limited

*Contract first: model output must conform
before use.*

VALIDATION GATES: PRE-CALL

Checks that run before the model is invoked:

-
-
-

VALIDATION GATES: PRE-CALL

Checks that run **before** the model is invoked:

- Input shape and required fields
-
-

VALIDATION GATES: PRE-CALL

Checks that run **before** the model is invoked:

- Input shape and required fields
- Size and type constraints
-

VALIDATION GATES: PRE-CALL

Checks that run **before** the model is invoked:

- Input shape and required fields
- Size and type constraints
- Policy constraints (allowed routes, feature flags)

VALIDATION GATES: PRE-CALL

Checks that run **before** the model is invoked:

- Input shape and required fields
- Size and type constraints
- Policy constraints (allowed routes, feature flags)

Failing here is cheap. Failing after a model call is not.

VALIDATION GATES: POST-CALL

Checks that run **after** the model responds:

-
-
-

VALIDATION GATES: POST-CALL

Checks that run **after** the model responds:

- Schema and parse validity
-
-

VALIDATION GATES: POST-CALL

Checks that run **after** the model responds:

- Schema and parse validity
- Policy checks (allowed categories, content rules)
-

VALIDATION GATES: POST-CALL

Checks that run **after** the model responds:

- Schema and parse validity
- Policy checks (allowed categories, content rules)
- Confidence threshold — trigger fallback if below minimum

VALIDATION GATES: POST-CALL

Checks that run **after** the model responds:

- Schema and parse validity
- Policy checks (allowed categories, content rules)
- Confidence threshold — trigger fallback if below minimum

Model output is untrusted until it passes post-call validation.

GATEWAY INTEGRATION GOALS

-
-
-
-

GATEWAY INTEGRATION GOALS

- Consistent model invocation contract across features
-
-
-

GATEWAY INTEGRATION GOALS

- Consistent model invocation contract across features
- Prompt and model provenance recorded in trace metadata
-
-

GATEWAY INTEGRATION GOALS

- Consistent model invocation contract across features
- Prompt and model provenance recorded in trace metadata
- Central routing and provider abstraction
-

GATEWAY INTEGRATION GOALS

- Consistent model invocation contract across features
- Prompt and model provenance recorded in trace metadata
- Central routing and provider abstraction
- Audit-ready request/response envelope

GATEWAY INTEGRATION GOALS

- Consistent model invocation contract across features
- Prompt and model provenance recorded in trace metadata
- Central routing and provider abstraction
- Audit-ready request/response envelope

The gateway is a governance control point, not just a transport wrapper.

FAILURE AND FALLBACK DESIGN

When validation fails:

-
-
-
-

FAILURE AND FALLBACK DESIGN

When validation fails:

- Do not return raw model output.
-
-
-

FAILURE AND FALLBACK DESIGN

When validation fails:

- Do not return raw model output.
- Return a deterministic "needs review" response.
-
-

FAILURE AND FALLBACK DESIGN

When validation fails:

- Do not return raw model output.
- Return a deterministic "needs review" response.
- Persist trace and failure reason for follow-up.
-

FAILURE AND FALLBACK DESIGN

When validation fails:

- Do not return raw model output.
- Return a deterministic "needs review" response.
- Persist trace and failure reason for follow-up.
- Maintain a UI-safe output contract at all times.

FAILURE AND FALLBACK DESIGN

When validation fails:

- Do not return raw model output.
- Return a deterministic "needs review" response.
- Persist trace and failure reason for follow-up.
- Maintain a UI-safe output contract at all times.

Safe fallback is intentional design — not a sign the feature failed.

TESTING STRATEGY FOR WORKFLOW MODULES

-
-
-
-

TESTING STRATEGY FOR WORKFLOW MODULES

- **Unit tests:** orchestration branches and decision logic
-
-
-

TESTING STRATEGY FOR WORKFLOW MODULES

- **Unit tests:** orchestration branches and decision logic
- **Contract tests:** request/response schema conformance
-
-

TESTING STRATEGY FOR WORKFLOW MODULES

- **Unit tests:** orchestration branches and decision logic
- **Contract tests:** request/response schema conformance
- **Validation tests:** rejection and fallback paths
-

TESTING STRATEGY FOR WORKFLOW MODULES

- **Unit tests:** orchestration branches and decision logic
- **Contract tests:** request/response schema conformance
- **Validation tests:** rejection and fallback paths
- **Integration tests:** gateway adapter stub with trace verification

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

-
-
-
-
-

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

- Accepts document input via a validated controller contract
-
-
-
-

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

- Accepts document input via a validated controller contract
- Runs deterministic workflow orchestration in a service layer
-
-
-

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

- Accepts document input via a validated controller contract
- Runs deterministic workflow orchestration in a service layer
- Calls the model via gateway with trace metadata and prompt version
-
-

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

- Accepts document input via a validated controller contract
- Runs deterministic workflow orchestration in a service layer
- Calls the model via gateway with trace metadata and prompt version
- Enforces post-call schema and policy checks
-

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

- Accepts document input via a validated controller contract
- Runs deterministic workflow orchestration in a service layer
- Calls the model via gateway with trace metadata and prompt version
- Enforces post-call schema and policy checks
- Returns an accepted output or a safe fallback response

MODULE 2 LAB BUILD TARGET

You will implement a NestJS endpoint that:

- Accepts document input via a validated controller contract
- Runs deterministic workflow orchestration in a service layer
- Calls the model via gateway with trace metadata and prompt version
- Enforces post-call schema and policy checks
- Returns an accepted output or a safe fallback response

Definition of done: each layer has a clear boundary and the fallback path is tested.

SUMMARY

- 1.
- 2.
- 3.
- 4.

SUMMARY

1. **Structure beats ad-hoc** calls for reliability and reuse.
- 2.
- 3.
- 4.

SUMMARY

1. **Structure beats ad-hoc** calls for reliability and reuse.
2. **Prompts are code artefacts** and must be versioned and reviewed.
- 3.
- 4.

SUMMARY

1. **Structure beats ad-hoc calls** for reliability and reuse.
2. **Prompts are code artefacts** and must be versioned and reviewed.
3. **Validation gates are mandatory** — pre-call and post-call.
- 4.

SUMMARY

1. **Structure beats ad-hoc calls** for reliability and reuse.
2. **Prompts are code artefacts** and must be versioned and reviewed.
3. **Validation gates are mandatory** — pre-call and post-call.
4. **Gateway integration** enables auditability and trace consistency.

BRIDGE TO MODULE 3

What we've built:

-

What's next:

-

-

BRIDGE TO MODULE 3

What we've built:

- A deterministic, traceable workflow pattern you can reuse across features.

What's next:

-
-

BRIDGE TO MODULE 3

What we've built:

- A deterministic, traceable workflow pattern you can reuse across features.

What's next:

- Decide when deterministic workflows are the right tool — and when they aren't.
-

BRIDGE TO MODULE 3

What we've built:

- A deterministic, traceable workflow pattern you can reuse across features.

What's next:

- Decide when deterministic workflows are the right tool — and when they aren't.
- Your first task in Module 3: map your workflow against the agent decision framework.

BRIDGE TO MODULE 3

What we've built:

- A deterministic, traceable workflow pattern you can reuse across features.

What's next:

- Decide when deterministic workflows are the right tool — and when they aren't.
- Your first task in Module 3: map your workflow against the agent decision framework.

Module 3 covers workflow vs agent design: predictability, tool usage, and where agentic behaviour is appropriate.

QUESTIONS?

Module 2 — Governed AI Feature Delivery